1. Work division for your team of 3



Member 1 — Voice Agent & Pipecat

Voice pipeline owner

Responsibilities

- Set up Pipecat and the microphone/audio pipeline.
- Integrate VAD, speech-to-text (STT), LLM and text-to-speech (TTS).
- Implement interruption handling and audio response playback.
- Measure candidate-stop-to-first-agent-audio latency.

- Test an accented-speech audio clip and report its WER.

Suggested files

- src/voice/pipeline.py
- src/voice/latency.py
- tests/test_voice_pipeline.py

Commits: voice pipeline, interruption handling, latency measurement and audio tests.



Member 2 — Interview Brain & State Management

Conversation logic owner

Responsibilities

- Load the resume and job description.
- Chunk content and attach section metadata.
- Extract candidate details, skills, projects and target role.
- Implement interview stages, question budget and time limit.
- Implement next_question(), save_answer() and end_interview().
- Generate grounded questions, evaluate answer completeness and manage follow-ups.

Suggested files

- src/ingestion/resume_loader.py
- src/interview/state.py
- src/interview/tools.py
- src/interview/question_engine.py

Commits: ingestion, state machine, tool calling, grounding and follow-up logic.



Member 3 — Evaluation, Persistence & Testing

Quality and reporting owner

Responsibilities

- Define the scoring rubric and evidence-based evaluator.
- Generate scores for each answer with transcript quotes.
- Calculate words per minute, long pauses and filler-word counts.

- Save and restore interview state and transcripts after every turn.
- Generate the final report and full transcript.
- Build six automated tests, score-stability checks and name-bias checks.

Suggested files

- src/evaluation/rubric.py
- src/evaluation/report.py
- src/persistence/session_store.py
- tests/test_acceptance.py
- tests/test_scoring_stability.py

Commits: rubric, report, persistence, automated tests and evaluation results.

My recommendation: If you are coordinating the project, take Member 2 — Interview Brain & State Management. It connects the resume, job description, conversation tools and interview stages, so you will understand the central logic of the whole application. If one teammate is more experienced with audio or testing, assign the other modules based on their strengths.

2. Shared GitHub repository structure

Use one repository with separate modules so everyone can work in parallel without constantly editing the same files.

```
ai-mock-interview-coach/
├── README.md
├── requirements.txt
├── .env.example
├── main.py
├── config.py
├── src/
│   ├── ingestion/
│   │   └── resume_loader.py
│   ├── voice/
│   │   ├── pipeline.py
│   │   └── latency.py
```

```
│   ├── interview/
│   │   ├── state.py
│   │   ├── tools.py
│   │   └── question_engine.py
│   ├── evaluation/
│   │   ├── rubric.py
│   │   └── report.py
│   └── persistence/
│       └── session_store.py
├── tests/
│   ├── test_acceptance.py
│   ├── test_voice_pipeline.py
│   └── test_scoring_stability.py
├── data/
│   └── sample_resume.txt
├── recordings/
└── reports/
```

Use fake or consented resume data and keep API keys in .env, which must be excluded from Git.

3. How to divide GitHub commits

Each member should commit their own implementation to a feature branch and open a pull request. Merge into main after reviewing and testing.

Member	Branch	Example commit messages
1	feature/voice-pipeline	feat: add Pipecat audio pipeline
1	Same branch	feat: add interruption handling and latency metrics
2	feature/interview-engine	feat: implement resume ingestion and chunk metadata

2	Same branch	feat: add interview state and follow-up tools
3	feature/evaluation-report	feat: implement evidence-based scoring rubric
3	Same branch	test: add safety, grounding and scoring stability tests

Don't create artificial commits just to increase the count. Each commit should represent a real, reviewable change.

4. Agree on interfaces before coding

This is the most important step for parallel development. All three members should agree on the function signatures and data formats before implementing their modules.

Member 1 → Member 2

Voice pipeline sends recognized speech and timestamps to the interview engine.

on_candidate_transcript(text: str, start_time: float, end_time: float)

Member 2 → Member 1

Interview engine returns the next response for the voice agent to speak.

process_candidate_answer(session_id: str, transcript: str) -> dict

Member 2 → Member 3

The interview engine provides a persisted session containing questions, answers, stages and timestamps.

load_session(session_id: str) -> dict

Member 3 → Main application

Evaluation consumes the saved session and generates a report without repeating the interview.

generate_report(session_id: str) -> dict

Treat these as proposed interfaces; document the final schemas in the repository before integration.

5. Suggested 7-day execution plan

Day 1

Repository and contracts

- Create the repository structure.
- Assign branches and module owners.
- Agree on state and transcript schemas.

Days 2–3

Parallel development

- Member 1 builds the voice pipeline.
- Member 2 builds ingestion and interview logic.
- Member 3 builds evaluation and persistence.

Days 4–5

Integration

- Connect voice to interview tools.
- Connect saved sessions to the evaluator.
- Verify that the interview ends within the configured budget.

Day 6

Acceptance tests

- Two grounding tests.

- Two follow-up tests.
- Two safety tests.
- Five-run scoring stability and name-bias checks.
- Accented-speech STT test and WER calculation.

Day 7

Final demonstration

- Run the full voice interview.
- Restart the application and restore a saved session.
- Generate the report and show the test results.
- Finalize README, setup instructions and architecture diagram.

6. Important grading checklist

Use this checklist as your team's definition of done.

Project completion

0/11

Voice pipeline

Pipecat microphone, VAD, STT, LLM, TTS and speaker integration
Interruptions stop agent speech; thinking pauses do not prematurely end turns
Per-turn latency and accented-audio WER are reported

Interview engine

Resume/JD ingestion with section metadata and summary
At least 3 resume-specific questions and 1 resume-to-JD question
Stateful stages, question/time limits, tools and maximum 2 follow-ups
Prompt injection treated as data; protected questions blocked

Evaluation and persistence

Every answer scored against a defined rubric with quote evidence
Speech metrics and full transcript included

Session restored after restart without repeating interview
Six acceptance tests, score spread across five runs and name-bias comparison

7. How to demonstrate individual contributions

Since commits influence the assessment, prepare evidence for each member:

- Member 1: show a live voice interaction, interruption handling and latency output.
- Member 2: show resume-grounded questions, state transitions and a weak-answer follow-up.
- Member 3: show a generated evidence-based report, restart recovery and the automated test table.

All three should understand the complete architecture and be able to explain their own code. Your instructor may also consider integration quality, testing, documentation and individual understanding, so check the official marking rubric rather than assuming commit count alone determines the marks.

One question: Have you already created the GitHub repository and committed some code, or is the repository currently empty? If you share the repository structure or link, I can divide the work around your actual files instead of making you reorganize everything.